Sequential RISC-V Processor Design

Project Phase I Report

Introduction to Processor Architecture
Spring 2026

Team Members

Team XX
Aryaman Saran (2024102027)
Sindhuja Babu (2024102055)
Lakshya Jindal (2023112017)

Institution
International Institute of Information Technology Hyderabad

Course
Introduction to Processor Architecture

February 22, 2026

1 Introduction

This project details the design, implementation, and verification of a sequential (non-pipelined) 64-bit RISC-V processor. Based on a specific subset of the RV64I Base Integer Instruction Set, the processor is designed to execute instructions in a single-cycle flow, ensuring that each instruction is fully completed before the next begins. The hardware description was authored in Verilog HDL and verified using the iVerilog simulation environment.

1.1 Project Objective

The primary goal was to bridge the gap between theoretical computer architecture and hardware realization by constructing a functional datapath. This involved implementing several critical modular components:

- **Program Counter (PC):** A 64-bit register responsible for instruction sequencing.
- **Instruction & Data Memory:** Byte-addressed memory modules configured for Big-Endian data storage.
- **Register File:** A 32-register bank of 64-bit wide registers.
- **Control Unit & ALU Control:** Logic blocks that decode opcodes and function fields to orchestrate the datapath.
- **Arithmetic Logic Unit (ALU):** The computational core performing arithmetic and logical operations.

1.2 Design Methodology

The processor utilizes a modular design approach. By coding each stage as a separate module, individual components could be independently tested for functionality before integration into the top-level architecture. This was particularly crucial for handling RISC-V ISA requirements, such as the Immediate Generation block, which performs sign-extension for multiple instruction formats (I-type, S-type, and B-type).

1.3 Scope of Implementation

The completed design successfully executes a foundational subset of RISC-V instructions, including:

- **Computational:** `add`, `sub`, `addi`, `and`, `or`
- **Memory Access:** `ld` (Load Doubleword), `sd` (Store Doubleword)
- **Control Flow:** `beq` (Branch if Equal)

2 Overall Implementation Overview

The processor is implemented as a modular sequential datapath integrating all required architectural components into a single top-level module. The design follows a sequential execution flow where each instruction passes through fetch, decode, execute, memory access, and write-back stages.

2.1 Top-Level Design Structure

The top-level module coordinates the interaction between the major functional blocks. The 64-bit Program Counter (PC) generates the instruction address, which is used by the Instruction Memory to fetch a 32-bit instruction combinationally. Instruction fields such as opcode, source registers, and destination register are extracted and used to control the datapath. Immediate values are generated through sign-extension logic to support multiple instruction formats.

2.2 Program Counter and Control Flow

The PC is updated on every positive clock edge. During normal execution, the PC increments by 4 to fetch the next instruction. For branch instructions, the PC is updated with a computed branch target address when the branch condition is satisfied, allowing controlled changes in program flow.

2.3 Control and Execution Strategy

The control unit generates the required control signals based on the instruction opcode, while the ALU control block determines the specific ALU operation using instruction function fields. These signals govern operand selection, ALU execution, memory access, and register write-back.

2.4 Memory Organization

The processor uses byte-addressable instruction and data memories following a Big-Endian format, as specified in the project requirements. Data memory supports load and store operations using control signals generated during execution.

2.5 Write-Back Mechanism

A multiplexer selects between ALU output and memory read data depending on the instruction type. The selected value is written back into the register file when register write is enabled.

2.6 Modular Implementation Approach

Each functional block was implemented as an independent Verilog module. This modular design enabled easier debugging and verification before integrating all components into the complete processor.

3 Program Counter and Instruction Fetch Stage

The Program Counter (PC) and Instruction Fetch stage together control the flow of instruction execution by generating instruction addresses, updating program flow, and fetching instructions from memory. The implementation was designed in a modular manner by separating the PC register, address computation logic, branch control logic, and instruction memory.

3.1 Program Counter (PC)

The Program Counter is implemented as a 64-bit register that stores the address of the current instruction. The PC is updated on the positive edge of the clock and reset asynchronously to zero when the reset signal is asserted.

- On reset, the PC is initialized to address zero.
- During normal execution, the PC is updated with the value provided through `pc_in`.

This design ensures synchronous progression of instructions while allowing external logic to control the next instruction address.

3.2 PC Update Logic

The next PC value is determined using separate combinational modules:

- **PC_adder:** Computes `PC + 4` to support sequential execution of 32-bit instructions.
- **PC_branchadder:** Computes the branch target address by adding the sign-extended immediate to the current PC value.

- **next_pc_mux:** Selects between sequential execution and branch execution. If both the Branch control signal and Zero condition are asserted, the branch target address is selected; otherwise, execution continues with `PC + 4`.

This modular structure clearly separates control-flow decision logic from the PC register itself, improving readability and simplifying debugging.

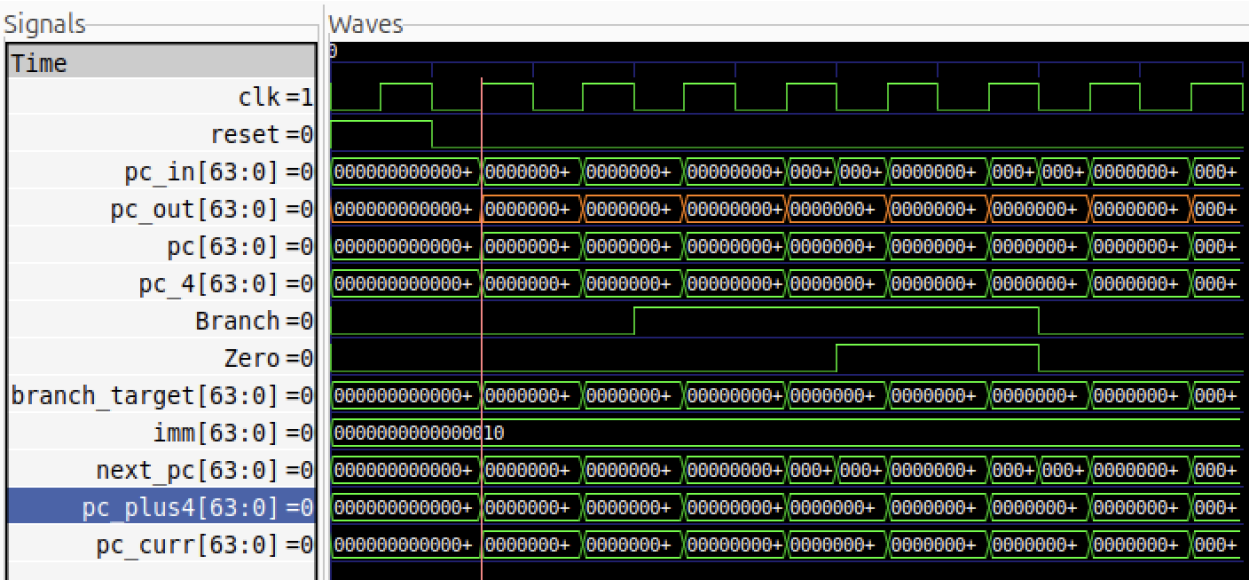

Figure 1: GTKWave waveform showing Program Counter updates and reset behaviour

3.3 Instruction Fetch Stage

The instruction fetch stage integrates the program counter and instruction memory into a single functional block. The current PC value is used as the address input to the instruction memory, which outputs a 32-bit instruction combinationally. The stage performs the following operations:

1. Update the PC based on branch decisions.
2. Compute sequential and branch target addresses.
3. Select the next PC value using a multiplexer.
4. Fetch the corresponding instruction from memory.

This implementation ensures continuous instruction flow while supporting control-flow changes through branch instructions.

3.4 Instruction Memory

Instruction memory is implemented as a byte-addressable array with a configurable size defined using `IMEM_SIZE`. Instructions are loaded from the external `instructions.txt` file using the `$readmemh` command. The fetched 32-bit instruction is constructed by concatenating four consecutive bytes:

```
{mem[addr], mem[addr+1], mem[addr+2], mem[addr+3]}
```

This follows a big-endian arrangement, where the lowest address corresponds to the most significant byte of the instruction.

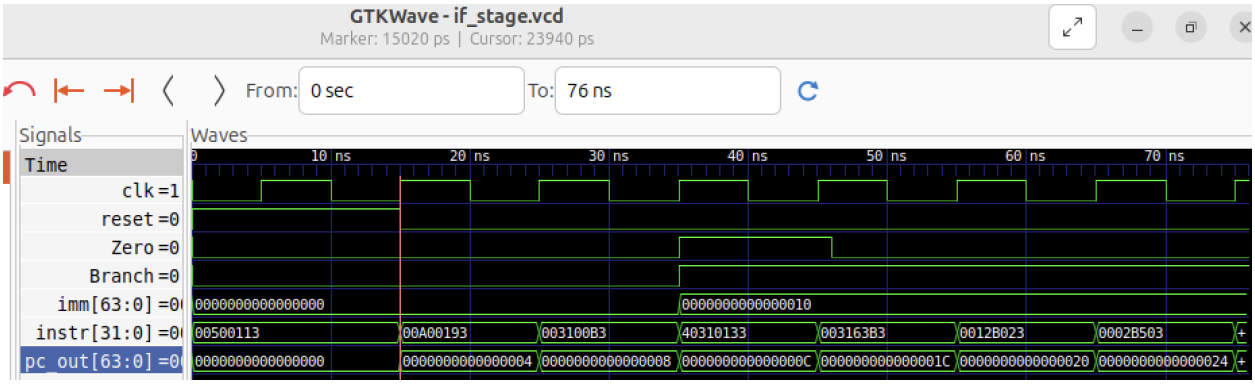

Figure 2: GTKWave waveform showing instruction fetch using PC address

3.5 Working Summary

During each cycle, the PC provides the instruction address to the instruction memory. The instruction is fetched combinationally, while the next PC value is determined using branch control signals and arithmetic units. On the next clock edge, the PC updates to the selected value, enabling sequential or branch-based execution flow.

4 Control and Execution Stage

The Control and Execution stage forms the decision-making and computational core of the processor datapath. This stage is responsible for decoding instructions, generating appropriate control signals, selecting operands, performing arithmetic and logical operations, and determining branch outcomes. The implementation follows a modular design consisting of the Control Unit, ALU Control Unit, operand selection logic, Arithmetic Logic Unit (ALU), and branch decision logic.

4.1 Instruction Field Extraction

The 32-bit instruction fetched from instruction memory is decoded to extract relevant control and execution fields:

- Opcode [6:0]: Determines the instruction category.
- funct3 [14:12]: Specifies the exact operation.
- funct7 [31:25]: Differentiates between operations such as addition and subtraction.

These fields are used by the Control Unit and ALU Control Unit to generate execution signals.

4.2 Control Unit

The Control Unit is implemented as combinational logic that generates control signals based on the instruction opcode. These signals coordinate datapath operation.

The generated signals include:

- RegWrite: Enables writing to register file.
- MemRead: Enables reading from data memory.
- MemWrite: Enables writing to data memory.
- MemtoReg: Selects memory output for write-back.

- ALUSrc: Selects ALU operand source.
- Branch: Indicates branch instruction.
- ALUOp [1:0]: Specifies ALU operation type.

Control signal behavior for supported instructions:

Instruction	RegWrite	ALUSrc	MemRead	MemWrite	Branch	ALUOp
add	1	0	0	0	0	10
sub	1	0	0	0	0	10
addi	1	1	0	0	0	10
and	1	0	0	0	0	10
or	1	0	0	0	0	10
ld	1	1	1	0	0	00
sd	0	1	0	1	0	00
beq	0	0	0	0	1	01

4.3 ALU Control Unit

The ALU Control Unit determines the specific ALU operation using ALUOp, funct3, and funct7 fields. It generates a 4-bit ALU opcode which directly controls the ALU.

Operation encoding:

Operation	ALU Opcode
ADD	0000
SUB	1000
AND	0111
OR	0110

For load and store instructions, addition is used to compute memory addresses. For branch instructions, subtraction is used to compare operands.

4.4 Operand Selection Logic

The ALUSrc multiplexer selects the second ALU operand:

- ALUSrc = 0: Register operand
- ALUSrc = 1: Immediate operand

This enables both register-register and register-immediate instructions.

4.5 Arithmetic Logic Unit Integration

The 64-bit ALU performs arithmetic and logical operations based on the ALU opcode. It produces:

- result [63:0]: Computed output
- zero flag: Indicates result equals zero
- carry and overflow flags: Provide arithmetic status

The ALU supports operations required by the RV64I subset implemented.

4.6 Branch Decision Logic

Branch decisions use the Branch signal and zero flag.

For the beq instruction, subtraction is performed. If the result equals zero, the branch condition is satisfied.

Branch decision logic:

$$\text{BranchTaken} = \text{Branch AND ZeroFlag}$$

This signal is used by PC update logic to control program flow.

4.7 Execution Flow Summary

During execution:

1. Instruction fields are decoded
2. Control Unit generates control signals
3. ALU Control Unit selects operation
4. Multiplexer selects operand
5. ALU executes operation
6. Branch logic determines branch outcome

4.8 Verification and Simulation

The Control and Execution stage was verified using Verilog simulation and GTKWave waveform analysis. The simulation confirmed correct generation of control signals, ALU operations, and branch decisions for all supported instructions including add, sub, addi, and, or, ld, sd, and beq.

Figure 3 shows the GTKWave simulation waveform. The ALUControl signal correctly changes based on instruction type, and the branchtaken signal is asserted only when the zeroflag is high for the beq instruction.

4.9 Working Summary

The Control and Execution stage correctly decodes instructions, generates control signals, executes ALU operations, and evaluates branch conditions. The modular design ensures reliable and scalable integration with the complete sequential processor datapath.

5 Register File

The Register File serves as the primary operand storage unit of the processor. It provides source operands for the execute stage and stores results produced during the write-back stage. The implemented design follows the RISC-V requirement of maintaining 32 general-purpose registers, each 64 bits wide.

5.1 Implementation Structure

The register file is implemented as an array of 32 registers:

```
reg [63:0] regs[0:31]
```

Two read ports and one write port are supported. The module accepts register indices for two source operands (`read_reg1`, `read_reg2`) and one destination register (`write_reg`), along with write data and a register write enable signal.

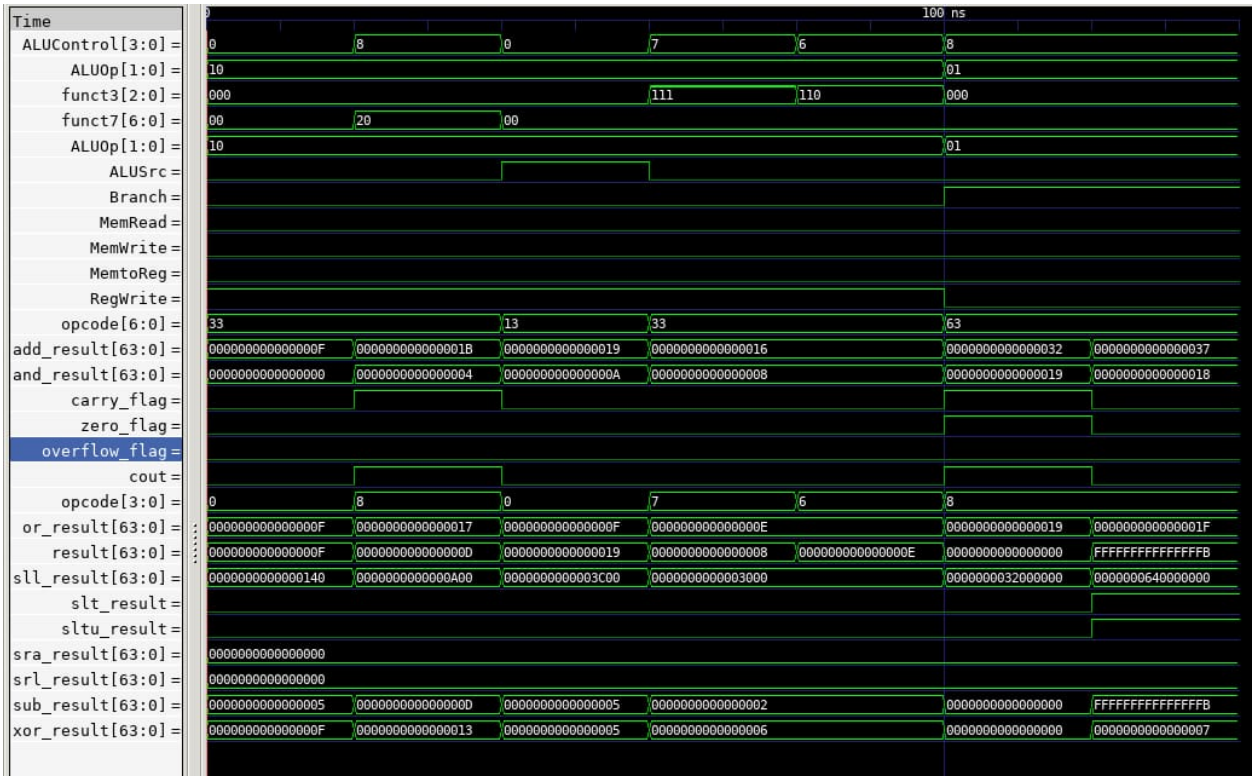

Figure 3: GTKWave waveform showing control signals, ALU control generation, and execution results

5.2 Write Operation

Write operations occur synchronously on the positive edge of the clock. When the `reg_write_en` signal is asserted, the value on `write_data` is written into the destination register.

- **Reset Behaviour:** When reset is asserted, all 32 registers are cleared to zero using a loop-based initialization.
- **Write Condition:** Writes are performed only when `reg_write_en` is high and the destination register is not zero.

The restriction on register zero ensures compatibility with the RISC-V convention where register `x0` remains constant.

5.3 Read Operation

Read operations are implemented combinationally using continuous assignments:

```
assign read_data1 = regs[read_reg1]
assign read_data2 = regs[read_reg2]
```

This allows both source operands to be accessed simultaneously within a clock cycle, enabling efficient datapath execution.

5.4 Working Summary

During execution, the register file provides two operand values based on the source register indices while simultaneously supporting a write-back operation on the clock edge. This dual-read and single-write behaviour allows arithmetic, memory, and branch instructions to access required operands without additional cycles.

6 Immediate Generation Unit

The Immediate Generation unit is responsible for extracting immediate fields from the fetched 32-bit instruction and converting them into 64-bit signed values. These immediate values are required for arithmetic instructions with immediates, memory addressing, and branch target computation. The implementation follows the RISC-V instruction format specifications while ensuring correct sign extension for 64-bit execution.

6.1 Implementation Approach

The immediate generation logic is implemented using combinational assignments. Based on the instruction opcode, the required immediate format is selected and sign-extended to 64 bits.

The design supports the following instruction formats:

- **I-type:** Used for instructions such as `addi` and `ld`.
- **S-type:** Used for store instructions such as `sd`.
- **B-type:** Used for branch instructions such as `beq`.

Each immediate type extracts specific bit fields from the instruction and performs sign extension so that negative offsets and arithmetic values are correctly represented.

6.2 Sign Extension

To maintain consistency with RV64I execution, immediates are extended from their original size to 64 bits by replicating the sign bit. This ensures correct behaviour during arithmetic operations and address calculations.

For example:

- I-type immediates are extracted from bits [31:20]
- S-type immediates combine bits [31:25] and [11:7]
- B-type immediates rearrange instruction fields and append a trailing zero for alignment

6.3 Working Summary

During execution, the immediate generation unit continuously monitors the instruction opcode and produces the corresponding sign-extended immediate value. This output is forwarded to the execute stage where it is used either as an ALU operand or for branch target computation.

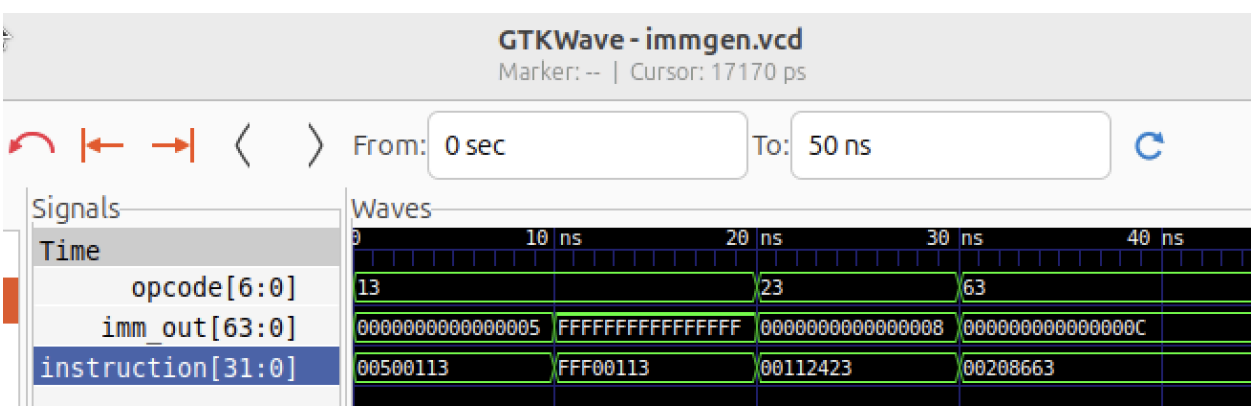

Figure 4: GTKWave waveform showing immediate extraction and sign-extension

7 Data Memory

The Data Memory module provides storage for load and store instructions. It is designed as a byte-addressable memory unit capable of reading and writing 64-bit data values. The implementation supports the `ld` (load doubleword) and `sd` (store doubleword) instructions of the RV64I subset.

7.1 Memory Organization

Data memory is implemented as an array of bytes:

```
reg [7:0] mem[0:DMEM_SIZE-1]
```

This structure ensures byte-level addressing, where each memory location stores one byte (8 bits). The memory size is parameterized, allowing flexible configuration during instantiation.

7.2 Write Operation (Store)

Write operations occur synchronously on the positive edge of the clock when the `MemWrite` signal is asserted. For store instructions, the 64-bit input data is divided into eight bytes and written into consecutive memory locations starting from the given base address.

- The store operation writes 8 consecutive bytes.
- The byte at the base address stores the most significant byte.
- Writes are performed only when the address is within valid memory bounds.

This behaviour implements the `sd` instruction.

7.3 Read Operation (Load)

Read operations are implemented combinationally. When `MemRead` is asserted, eight consecutive bytes starting from the base address are concatenated to form a 64-bit output value.

- The load operation reads 8 consecutive bytes.
- Data is assembled into a 64-bit word before being forwarded to the write-back stage.

This corresponds to the behaviour of the `ld` instruction.

7.4 Big-Endian Data Format

The memory follows a Big-Endian organization, where the lowest memory address stores the most significant byte of the 64-bit word. This design choice aligns with the project specifications and is consistently maintained across both read and write operations.

7.5 Working Summary

During execution, the ALU provides the memory address while control signals determine whether a load or store operation is performed. Store instructions write eight bytes into memory on the clock edge, whereas load instructions read eight bytes combinationally and forward the assembled 64-bit value for write-back.

8 Supported Features

The implemented processor supports a functional subset of the RV64I instruction set architecture. The design integrates instruction fetch, decode, execution, memory access, and write-back functionality within a sequential datapath. The following features were successfully implemented and verified through simulation.

8.1 Instruction Set Support

The processor executes multiple categories of instructions:

- **Arithmetic and Logical Instructions:** `add`, `sub`, `addi`, `and`, `or`
- **Memory Instructions:** `ld` (load doubleword), `sd` (store doubleword)
- **Control Flow Instructions:** `beq` (branch if equal)

These instructions cover computational operations, memory access, and basic control flow required for sequential execution.

8.2 64-bit Datapath

The processor follows the RV64I architecture using a fully 64-bit datapath:

- 64-bit Program Counter
- 64-bit register file entries
- 64-bit ALU operations
- 64-bit memory read and write paths

This enables execution of doubleword operations and consistent arithmetic behaviour across all stages.

8.3 Register File Features

The register file supports two simultaneous read ports and one synchronous write port. Register `x0` is protected from modification, ensuring compliance with the RISC-V ISA convention. Reads are combinational, enabling operand availability within the same cycle.

8.4 Immediate Generation Support

Immediate values are generated for multiple instruction formats:

- I-type immediates for arithmetic and load instructions
- S-type immediates for store instructions
- B-type immediates for branch instructions

All immediates are sign-extended to 64 bits before being used in execution.

8.5 Branch Handling

Branch operations are supported through dedicated branch decision logic. The processor evaluates branch conditions using ALU comparison results, and updates the program counter with a branch target address when the condition is satisfied.

8.6 Memory Organization

Both instruction memory and data memory are implemented as byte-addressable arrays following a Big-Endian format. The memory system supports:

- Loading 64-bit values by reading eight consecutive bytes
- Storing 64-bit values by writing eight consecutive bytes
- Combinational memory reads and synchronous writes

8.7 Execution Control Features

The control unit generates datapath control signals based on opcode decoding, while the ALU control logic determines the exact ALU operation from instruction function fields. Operand selection through multiplexers allows both register-based and immediate-based execution.

8.8 Halt Detection

A simple halt mechanism is implemented at the top-level design. Execution stops when an all-zero instruction or an ECALL instruction (`0x00100073`) is detected, preventing further PC updates.

8.9 Simulation and Verification

All supported features were verified using Verilog simulation and GTKWave waveform analysis. Simulations confirmed correct instruction fetch, control signal generation, ALU execution, register updates, memory access, and branch behaviour.

9 Challenges Encountered

During development, a few implementation issues were encountered while integrating the different processor modules. The primary issue observed was incorrect register output (notably register `x13`), which was traced to the instruction fetch stage where the Program Counter was not updating correctly. This caused repeated instruction execution until the PC update logic was fixed.

Additional challenges included verifying correct branch behaviour through the next-PC multiplexer logic, ensuring proper Big-Endian byte ordering during instruction and data memory accesses, and debugging immediate generation for branch offsets. Since most modules interacted closely, small issues in one stage often affected later stages.

Debugging was mainly performed using Verilog debug print statements to track PC values, instructions, immediates, and register updates during simulation.